AI-CAD House Builder: A Tool-Using Large Language Model Framework for Natural-Language Architectural CAD

Shyamsundar Kamalakannan[1], Dhasaradhan D[2], Prajeshwaran B[3], G. Revathy[4]
[4]Associate Professor,[1,2,3]UG Scholar
Department of Artificial Intelligence and Machine Learning, Vels Institute of Technology Pallavaram ,India
Email id: sundarshyam5780@gmail.com,prajeshpraju2006@gmail.com, dhasaradhandevaranjith@gmail.com , revathy.se@vistas.ac.in

***Abstract*—Conventional architectural computer-aided design (CAD) systems provide precise and editable geometry, but they require users to understand commands, dimensions, object relationships, and modeling workflows. Recent research in text-to-CAD, CAD agents, verification, and architectural floor-plan generation shows that large language models (LLMs) can reduce this interaction barrier, but direct generation of complete CAD geometry remains difficult to validate and edit reliably. This paper presents AI-CAD House Builder, a lightweight web-based architectural CAD framework that combines natural-language interaction, a semantic architectural tool registry, deterministic geometry generation, editable 2D floor plans, and interactive 3D visualization. The central design treats a local LLM as a planner rather than as the geometry engine. A Qwen2.5 7B Instruct model served through llama.cpp interprets user requirements and proposes restricted structured operations. A validation layer checks parameters, object references, dimensions, and basic geometric constraints before a deterministic CAD layer updates the model. The system supports hybrid manual and AI interaction so users can create or modify a design without regenerating the complete house. A literature-driven evaluation protocol is defined using tool success, object completion, dimensional error, constraint violations, edit success, latency, and manual correction effort. The contribution is a reproducible architecture for studying small local LLMs as controllable architectural CAD planners while preserving editable geometry and user control.**

***Index Terms*—architectural CAD, large language models, text-to-CAD, CAD agents, floor-plan generation, tool calling, deterministic geometry, natural-language interface**

I. Introduction

Computer-aided design (CAD) is widely used to create precise architectural drawings and three-dimensional building models. Its precision is valuable, but traditional CAD interaction assumes that the user knows commands, coordinate systems, dimensions, object relationships, and the correct order of operations. A user may know the desired house in natural language while still needing substantial CAD knowledge to convert that intent into walls, rooms, doors, windows, floors, and a consistent three-dimensional model.

Large language models (LLMs) provide a potential natural-language interface to technical software. Instead of requiring users to remember command sequences, an LLM can interpret design intent and select a sequence of structured operations. Recent text-to-CAD systems such as Text2CAD [1] and CAD-Llama [2] demonstrate language-conditioned generation of structured CAD designs. CADFusion [3] shows that visual feedback can improve language-driven CAD generation, while CADCodeVerify [4] emphasizes verification and iterative refinement. More recently, CAD-Assistant [5] and TOOLCAD [6] investigate tool-using LLM agents that interact with external CAD environments.

Architectural floor-plan research provides a second foundation. House-GAN [8], House-GAN++ [9], Graph2Plan [10], WallPlan [11], and HouseDiffusion [12] demonstrate that room relationships, wall structure, and geometric constraints are central to valid floor-plan generation. HouseTune [13] separates language-level planning from geometric refinement, while HouseMind [14] focuses on semantic representation and editing of architectural floor plans. MANSION [15] extends language-driven generation toward multi-floor building-scale 3D scenes.

Despite these advances, a practical gap remains for a lightweight interactive architectural CAD environment in which a small locally deployed LLM is restricted to semantic CAD operations while deterministic software retains control of geometry and persistent model state. Directly asking a small LLM to generate the entire geometric state is fragile: syntactically valid output can contain missing objects, invalid dimensions, incorrect references, or inconsistent relationships. The base project motivating this work therefore defines the LLM as a planner and the geometry engine as a deterministic executor.

This paper presents AI-CAD House Builder, a web-based prototype implementing that principle for a focused single-floor residential domain. The main contributions are:

1) a semantic architectural tool registry that constrains the actions available to the LLM;
2) a planner–validator–geometry architecture that separates probabilistic language reasoning from deterministic CAD execution;
3) a persistent object model supporting hybrid manual and natural-language editing;
4) an integrated 2D floor-plan and interactive 3D visualization workflow; and
5) a reproducible evaluation protocol for measuring tool reliability, geometric correctness, editing success, and user effort.

II. Related Work

A. Natural-Language-to-CAD Generation

Text2CAD [1] established a large-scale direction for generating sequential CAD designs from text prompts and evaluates visual quality, parametric precision, and geometric accuracy. CAD-Llama [2] uses a structured parametric CAD representation to expose CAD-specific knowledge to a language model. These works support the use of structured representations rather than treating CAD as an unstructured image-generation problem.

Text-to-CadQuery [7] explores direct natural-language generation of executable CadQuery code. Such code generation is flexible, but unrestricted code execution makes parameter validation and persistent state management more difficult for a small interactive application. AI-CAD House Builder therefore adopts a narrower typed tool interface rather than arbitrary code generation.

B. Tool-Using CAD Agents

CAD-Assistant [5] presents a tool-augmented VLLM architecture in which a model acts as a planner and interacts with CAD-specific tools while observing the evolving geometry. TOOLCAD [6] frames text-to-CAD as an interactive long-horizon tool-use problem and studies feedback and reinforcement learning. These studies are particularly relevant because the proposed system also treats CAD operations as explicit actions.

The distinction in this work is the target domain and deployment constraint. Rather than building a general-purpose mechanical CAD agent, the system uses a compact architectural vocabulary: CreateBoundary, CreateRoom, CreateWall, CreateDoor, CreateWindow, Move, Resize, Delete, and related visualization operations. This smaller action space enables deterministic validation and a simpler demonstration environment.

C. Verification and Feedback

CADCodeVerify [4] investigates iterative verification of generated CAD code using vision-language models. CADFusion [3] incorporates visual feedback because a sequence can be syntactically correct while producing an undesirable result. These studies motivate a post-action validation boundary in the proposed system.

In AI-CAD House Builder, validation initially operates on structured model state and geometric rules rather than a learned visual critic. This is deliberate: deterministic checks are easier to reproduce and do not require a second large model. A future version can add rendered-image feedback as an additional verification stage.

D. Architectural Floor-Plan Generation

House-GAN [8] and House-GAN++ [9] model relationships among rooms for layout synthesis and refinement. Graph2Plan [10] represents the floor plan using a layout graph and boundary, while WallPlan [11] focuses on wall-graph generation. HouseDiffusion [12] explicitly models vector floor-plan geometry and relationships such as orthogonality and corner sharing.

These methods demonstrate that architectural validity depends on topology and geometry, not only on semantic room names. The proposed system therefore stores room, wall, and opening relationships explicitly and validates operations against the current model.

HouseTune [13] uses an LLM-assisted initial layout followed by geometric refinement. HouseMind [14] introduces semantic floor-plan representations that support understanding, generation, and editing. MANSION [15] addresses long-horizon language-driven multi-floor 3D scene generation. The current work is intentionally narrower: the implementation focuses on single-floor houses so that the planner–tool–validator mechanism can be evaluated before introducing multi-floor structural dependencies.

III. Research Gap and Design Goals

The literature indicates three recurring challenges. First, language generation and geometric validity have different reliability requirements. Second, architectural design depends on semantic relationships such as room adjacency, wall ownership, and opening placement. Third, useful CAD assistance requires persistent editing rather than one-shot generation.

The design goals are therefore:

1) **Controllability:** the LLM must operate only through a defined architectural tool set.
2) **Validity:** invalid dimensions, references, and unsupported operations must be rejected before geometry execution.
3) **Editability:** generated objects must remain ordinary application objects that can be selected, moved, resized, or deleted.
4) **Statefulness:** follow-up natural-language instructions must target the current model instead of recreating it.
5) **Local deployment:** the planner should be executable locally using a compact quantized LLM.
6) **Reproducibility:** experiments must use fixed prompts, known initial states, explicit metrics, and recorded tool traces.

IV. Methodology

A. System Architecture

Fig. 1 summarizes the proposed architecture. The user enters a natural-language request or performs a manual CAD operation. The natural-language request is sent to the local planner. The planner receives a compact model summary and the allowed tool definitions. It returns structured tool calls. The validator checks each call against a schema and the current model. Valid calls are executed by deterministic geometry functions. The resulting model state is rendered in both 2D and 3D.

TABLE I
LITERATURE SYNTHESIS AND DESIGN IMPLICATIONS FOR AI-CAD HOUSE BUILDER

Work	Venue	Core idea and relevant finding	Design implication for this project
Text2CAD [1]	NeurIPS 2024	Natural-language prompts are mapped to sequential CAD designs, with evaluation of visual quality, parametric precision, and geometric accuracy.	Use structured CAD operations rather than unrestricted complete-geometry generation.
CAD-Llama [2]	CVPR 2025	Uses a structured parametric CAD representation to expose CAD-specific knowledge to a language model.	Maintain a compact semantic representation between language planning and deterministic geometry.
CADFusion [3]	ICML 2025	Adds visual feedback to sequential text-to-CAD generation because a valid sequence can still produce undesirable geometry.	Add post-execution validation and reserve visual feedback for a future verification stage.
CADCodeVerify [4]	ICLR 2025	Uses iterative verification and refinement for generated CAD code.	Treat model-generated operations as proposed actions that must pass validation before execution.
CAD-Assistant [5]	ICCV 2025	Uses a VLLM planner, CAD-specific tools, and evolving geometry feedback.	Directly motivates the planner–tool-registry architecture used here.
TOOLCAD [6]	ACL Findings 2026	Frames text-to-CAD as long-horizon tool use with feedback and post-training.	Support explicit tool interfaces, multi-step plans, and state-aware execution.
House-GAN [8]	ECCV 2020	Represents room relationships as graph constraints for house-layout generation.	Store room relationships explicitly rather than relying only on room names.
House-GAN++ [9]	CVPR 2021	Performs graph-constrained layout refinement for professional architectural use.	Support incremental edits instead of regenerating an entire house.
Graph2Plan [10]	SIGGRAPH 2020	Uses a layout graph and boundary to generate floor plans under sparse user constraints.	Represent room adjacency and external-boundary constraints explicitly.
WallPlan [11]	SIGGRAPH 2022	Generates wall graphs with room labels to synthesize structured floor plans.	Treat walls as first-class connected objects with persistent identities.
HouseDiffusion [12]	CVPR 2023	Generates vector floor plans while modeling orthogonality, parallelism, and corner sharing.	Encode and validate basic geometric relationships in the deterministic CAD layer.
HouseTune [13]	AAAI 2026	Uses LLM assistance for initial layout planning followed by geometric refinement.	Keep semantic planning and geometric realization as separate stages.
HouseMind [14]	CVPR 2026	Uses semantic floor-plan representations for architectural understanding, generation, and editing.	Maintain semantic room/object identities so follow-up language can target existing objects.
MANSION [15]	CVPR 2026	Addresses language-driven multi-floor, building-scale 3D generation and long-horizon editing.	Motivate future multi-floor support; keep the present evaluation single-floor for controlled experiments.
Unified Vector Floorplan [18]	CVPR 2026	Introduces Floorplan Markup Language (FML), a unified structured grammar for diverse vector floor-plan conditions.	A compact, structured floor-plan schema can improve controllability and consistent model-state exchange.

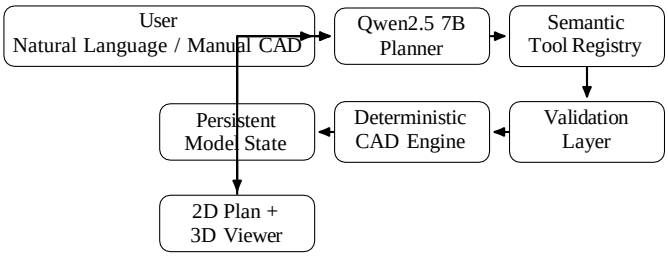


Fig. 1. Planner–tool–validator–geometry architecture of AI-CAD House Builder.

B. Requirement Extraction

The planner identifies explicit requirements from a user request, including plot dimensions, room counts, room names, approximate sizes, openings, and modification targets. The planner is instructed not to invent major missing requirements. If a request is ambiguous or contradictory, it should produce a clarification action rather than execute an unsafe assumption.

C. Semantic Tool Registry

The system exposes a restricted set of architectural actions. Representative operations are:

- `create_boundary(width,length)`
- `create_room(name,width,length,x,y)`
- `create_wall(start,end,thickness)`
- `create_door(room,wall,width,position)`
- `create_window(room,wall,width,position)`
- `move_object(id,dx,dy)`
- `resize_room(id,width,length)`
- `delete_object(id)`

The tool schema is deliberately smaller than a professional CAD command set. The purpose is not to reproduce every low-level CAD command but to expose meaningful architectural actions that can be evaluated.

D. Structured Tool Calls

For a request such as “Add a 1.2 m window to the north wall of Bedroom 2,” the planner should return a compact operation such as:

```
{
  "tool": "create_window",
  "parameters": {
    "room": "Bedroom 2",
    "wall": "north",
    "width": 1.2
  }
}
```

The language model does not calculate the wall coordinates. The application resolves the room and wall from persistent state, determines an available location, and performs the geometric operation.

E. Validation Layer

Validation is applied before execution. The validator checks:

1) whether the requested tool exists;
2) whether all required parameters are present;
3) whether numeric parameters are within supported ranges;
4) whether referenced objects exist;
5) whether a door or window is attached to a compatible wall;
6) whether the requested opening fits on the wall;
7) whether a room resize preserves the supported boundary constraints; and
8) whether the resulting state violates defined basic geometry rules.

A failed validation returns a structured error. The model is not allowed to directly modify geometry outside the registry.

F. Persistent Semantic Model

Each object has a stable identifier. A wall stores its endpoints, thickness, height, and relationships to attached openings. A room stores its semantic name, boundary, area, and wall references. A door or window stores its parent wall and position along that wall. This representation enables a later request such as "make Bedroom 2 wider" to locate the existing room.

G. 2D and 3D Visualization

The web front end uses React and Vite. The 2D interface provides CAD-like editing of walls, rooms, doors, and windows. The 3D representation uses Three.js through React Three Fiber. Because the 3D scene is derived from the same semantic model as the 2D plan, an edit to the model can update both representations.

H. Local AI Deployment

The prototype uses Qwen2.5 7B Instruct in quantized Q4_K_M GGUF form through llama.cpp. The local server provides an OpenAI-compatible chat-completion endpoint. Local inference is useful for a student research prototype because it reduces dependence on a remote API and permits repeatable experiments under controlled settings. The current deployment uses CPU inference on the available development hardware.

V. Prototype Implementation

The prototype is implemented as a browser-based application with a React/Vite front end and a Python/FastAPI back end. The interface includes the 2D plan editor, 3D viewer, object interaction, property editing, save/load support, and export functionality. The back end is responsible for AI orchestration and model-state services.

The implementation deliberately preserves the separation between AI and geometry. A planner response is treated as a proposed operation rather than as authoritative model state. This design also addresses a practical issue observed during development: a 7B language model can generate JSON wrapped in Markdown fences or omit expected fields even when its textual answer is correct. Parsing and validation therefore occur before any geometry operation.

The current prototype scope is single-floor residential design. A representative baseline contains a 10 m × 8 m house with bedrooms, living space, kitchen, bathrooms, doors, and windows. The single-floor restriction is an experimental boundary rather than a claim that the framework cannot support multiple levels. Multi-floor planning, stairs, balconies, and roof relationships are future extensions.

TABLE II
Evaluation Scenarios

ID	Prompt type	Expected behavior
T1	Simple two-bedroom house	Create requested rooms and basic walls.
T2	Three-bedroom house	Create rooms, openings, and dimensions.
T3	Two-floor request	Report unsupported scope safely in current prototype.
T4	Feature addition	Modify existing model without rebuilding unrelated objects.
T5	Room resize	Change target room while preserving external boundary.
T6	Conflicting dimensions	Ask for clarification or reject safely.
T7	Invalid reference	Reject unknown object without corrupting state.

VI. Experimental Protocol

A. Research Questions

The evaluation is organized around four research questions:

- RQ1: Can a local LLM reliably translate house-design language into valid semantic tool calls?
- RQ2: Does validation reduce invalid or unsafe CAD operations?
- RQ3: Can the persistent semantic model support targeted follow-up edits without regenerating unrelated geometry?
- RQ4: How much manual correction is required after AI-assisted generation?

B. Benchmark Scenarios

Seven core scenarios are derived from the base project specification:

C. Metrics

Tool Success Rate (TSR) is defined as

$$TSR = \frac{N_{\text{successful tool executions}}}{N_{\text{attempted tool executions}}}. \quad (1)$$

Object Completion (OC) is

$$OC = \frac{N_{\text{correctly created requested objects}}}{N_{\text{requested objects}}}. \quad (2)$$

For a requested dimension d_r and generated dimension d_g, absolute dimensional error is

$$E_d = |d_r - d_g|. \quad (3)$$

Constraint violations are counted across a fixed set of checks, including invalid object references, impossible opening placement, disconnected wall conditions, unsupported dimensions, and defined room-boundary violations.

Edit Success (ES) is the proportion of follow-up instructions that modify the intended existing object without unintended modifications to unrelated objects.

Time to Result (TTR) is measured from request submission to the first usable validated model state. Manual Effort (ME) is measured as the number of manual corrective operations required after AI execution.

D. Baselines and Ablations

A useful controlled comparison consists of:

1) **Direct generation:** LLM attempts to return complete house geometry/state.
2) **Planner without validation:** LLM returns semantic tool calls that are executed without the validation layer.
3) **Planner with validation:** the proposed architecture.

The comparison should use identical prompts, initial model states, and inference settings. At least five repeated trials per prompt are recommended for a student-scale study. Numerical results must be reported only after these runs are completed.

VII. Results and Discussion

The present manuscript distinguishes implementation observations from measured experimental results. The prototype has been integrated with a local llama.cpp server and a Qwen2.5 7B Instruct model, and direct model requests demonstrate that the model can respond to structured prompts. However, the development process also showed that complete-house JSON generation is not sufficiently reliable as the sole source of geometric state: model output can be wrapped in Markdown code fences or omit fields expected by a strict parser. This observation motivates the planner-to-tool architecture, but it is not a quantitative benchmark result.

The expected effect of the proposed validation boundary is a reduction in invalid operations and a clearer separation between language errors and geometry errors. The persistent model should also reduce the need for full regeneration during follow-up edits. These claims must be tested with the ablation protocol in Section V before being reported as measured improvements.

Table III is intentionally a results template rather than fabricated data. It specifies the exact quantities that should be filled from experiment logs.

From a systems perspective, the architecture provides three benefits. First, the semantic tool vocabulary limits the action space available to the local model. Second, validation creates an explicit safety and correctness boundary before geometry changes. Third, persistent identifiers enable incremental editing, which is important because architectural design is inherently iterative. These properties align with the planner-and-tool direction demonstrated by CAD-Assistant and TOOLCAD [5], [6], while the semantic room and wall structure is motivated by architectural floor-plan research [9]–[12].

TABLE III
Experimental Results Template

Metric	Direct	No Val.	Proposed	Unit
TSR	–	–	–	%
Object Completion	–	–	–	%
Dimensional Error	–	–	–	m
Constraint Violations	–	–	–	count
Edit Success	–	–	–	%
Time to Result	–	–	–	s
Manual Effort	–	–	–	operations

A limitation is that the current validator is rule-based and does not yet perform professional architectural analysis. A geometrically valid room arrangement may still be functionally poor. A future visual evaluator could follow the feedback principles explored by CADFusion [3] and CADCodeVerify [4]. Another limitation is the single-floor scope. Multi-floor reasoning introduces additional constraints for stairs, vertical circulation, slabs, and alignment; MANSION [15] demonstrates why long-horizon building-scale generation is substantially more complex.

VIII. Threats to Validity and Limitations

The prototype is not a replacement for professional architectural CAD or regulatory review. It does not certify structural safety, building-code compliance, electrical safety, plumbing compliance, or construction readiness.

The benchmark is also intentionally small. Seven scenarios can demonstrate system behavior, but they cannot establish generalization across all residential layouts. Larger prompt sets and independently authored requirements would be needed for stronger claims.

Results from published generative floor-plan papers should not be compared directly with the proposed application-level metrics. Those works often use large benchmark datasets and learned generative models, whereas this work evaluates a software architecture and interactive tool-use pipeline. A fair comparison would require a common dataset, common prompts, and carefully aligned metrics.

Finally, local LLM behavior can vary with quantization, inference settings, prompt construction, and available context. The final experiment report should record the model identifier, quantization, context size, temperature, maximum output length, hardware, and software versions.

IX. Conclusion

AI-CAD House Builder presents a practical framework for natural-language architectural CAD in which a local LLM performs planning while deterministic software retains control of geometry and persistent model state. The approach is motivated by recent advances in text-to-CAD generation, tool-using CAD agents, verification, and architectural floor-plan synthesis.

The central contribution is not unrestricted automatic geometry generation. Instead, the system defines a controlled

semantic interface between language and CAD. The planner selects operations such as creating rooms, walls, doors, and windows; the validator checks those operations; and the geometry engine produces editable model state that can be visualized in both 2D and 3D. This architecture is particularly suitable for a compact locally deployed LLM because it reduces the amount of geometric reasoning expected from the language model.

Future work includes a visual geometry critic, stronger room-adjacency and constraint optimization, multi-floor generation, BIM-oriented semantics, richer CAD export, architectural template retrieval, and larger human-authored evaluation sets. The next experimental milestone is to execute the defined benchmark and ablation study and replace the results placeholders with measured values and failure analysis.

REFERENCES

[1] M. S. Khan et al., "Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 7552–7579.

[2] J. Li et al., "CAD-Llama: Leveraging Large Language Models for Computer-Aided Design Parametric 3D Model Generation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 18563–18573.

[3] R. Wang, Y. Yuan, S. Sun, and J. Bian, "Text-to-CAD Generation Through Infusing Visual Feedback in Large Language Models," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, PMLR 267, 2025, pp. 65326–65345.

[4] K. Alrashedy et al., "Generating CAD Code with Vision-Language Models for 3D Designs," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[5] D. Mallis et al., "CAD-Assistant: Tool-Augmented VLLMs as Generic CAD Task Solvers," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025, pp. 7284–7294.

[6] Y. Gong, X. Wu, W. Liu, and Tukang, "TOOLCAD: Exploring Tool-Using Large Language Models in Text-to-CAD Generation with Reinforcement Learning," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 23161–23188.

[7] H. Xie and F. Ju, "Text-to-CadQuery: A New Paradigm for CAD Generation with Scalable Large Model Capabilities," arXiv:2505.06507, 2025.

[8] N. Nauata et al., "House-GAN: Relational Generative Adversarial Networks for Graph-Constrained House Layout Generation," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 162–177.

[9] N. Nauata et al., "House-GAN++: Generative Adversarial Layout Refinement Network towards Intelligent Computational Agent for Professional Architects," in *Proc. IEEE/CVF CVPR*, 2021, pp. 13632–13641.

[10] R. Hu, Z. Huang, Y. Tang, O. Van Kaick, H. Zhang, and H. Huang, "Graph2Plan: Learning Floorplan Generation from Layout Graphs," *ACM Trans. Graphics*, vol. 39, no. 4, 2020.

[11] J. Sun et al., "WallPlan: Synthesizing Floorplans by Learning to Generate Wall Graphs," *ACM Trans. Graphics*, vol. 41, no. 4, 2022.

[12] M. A. Shabani, S. Hosseini, and Y. Furukawa, "HouseDiffusion: Vector Floorplan Generation via a Diffusion Model With Discrete and Continuous Denoising," in *Proc. IEEE/CVF CVPR*, 2023, pp. 5466–5475.

[13] Z. Zong et al., "HouseTune: Two-Stage Floorplan Generation with LLM Assistance," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 40, no. 16, 2026, pp. 14059–14067.

[14] S. Qin, R. E. Weber, and X. Lu, "Tokenization Allows Multimodal Large Language Models to Understand, Generate and Edit Architectural Floor Plans," in *Proc. IEEE/CVF CVPR*, 2026, pp. 10430–10440.

[15] L. Che et al., "MANSION: Multi-floor lANguage-to-3D Scene generatIOn for loNg-horizon tasks," in *Proc. IEEE/CVF CVPR*, 2026, pp. 37175–37185.

[16] C. Li, H. Pan, A. Bousseau, and N. J. Mitra, "Free2CAD: Parsing Freehand Drawings into CAD Commands," *ACM Trans. Graphics*, vol. 41, no. 4, 2022.

[17] R. Wu, C. Xiao, and C. Zheng, "DeepCAD: A Deep Generative Network for Computer-Aided Design Models," in *Proc. IEEE/CVF ICCV*, 2021, pp. 6772–6782.

[18] K. Shiohara and T. Yamasaki, "Unified Vector Floorplan Generation via Markup Representation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2026, pp. 39262–39271.

[19] W. Ma et al., "Draw Step by Step: Reconstructing CAD Construction Sequences from Point Clouds via Multimodal Diffusion," in *Proc. IEEE/CVF CVPR*, 2024, pp. 27154–27163.